VELOOP REWARDS - Data Pipeline Proposal

Data Analyst Internship, Task 01 | Work Package O

Authenticity notice: This is a **Proposed** design. Nothing is claimed to exist in VELOOP, no threshold is invented (limits are 'to be agreed'), and no simulated or real VELOOP statistics appear here.

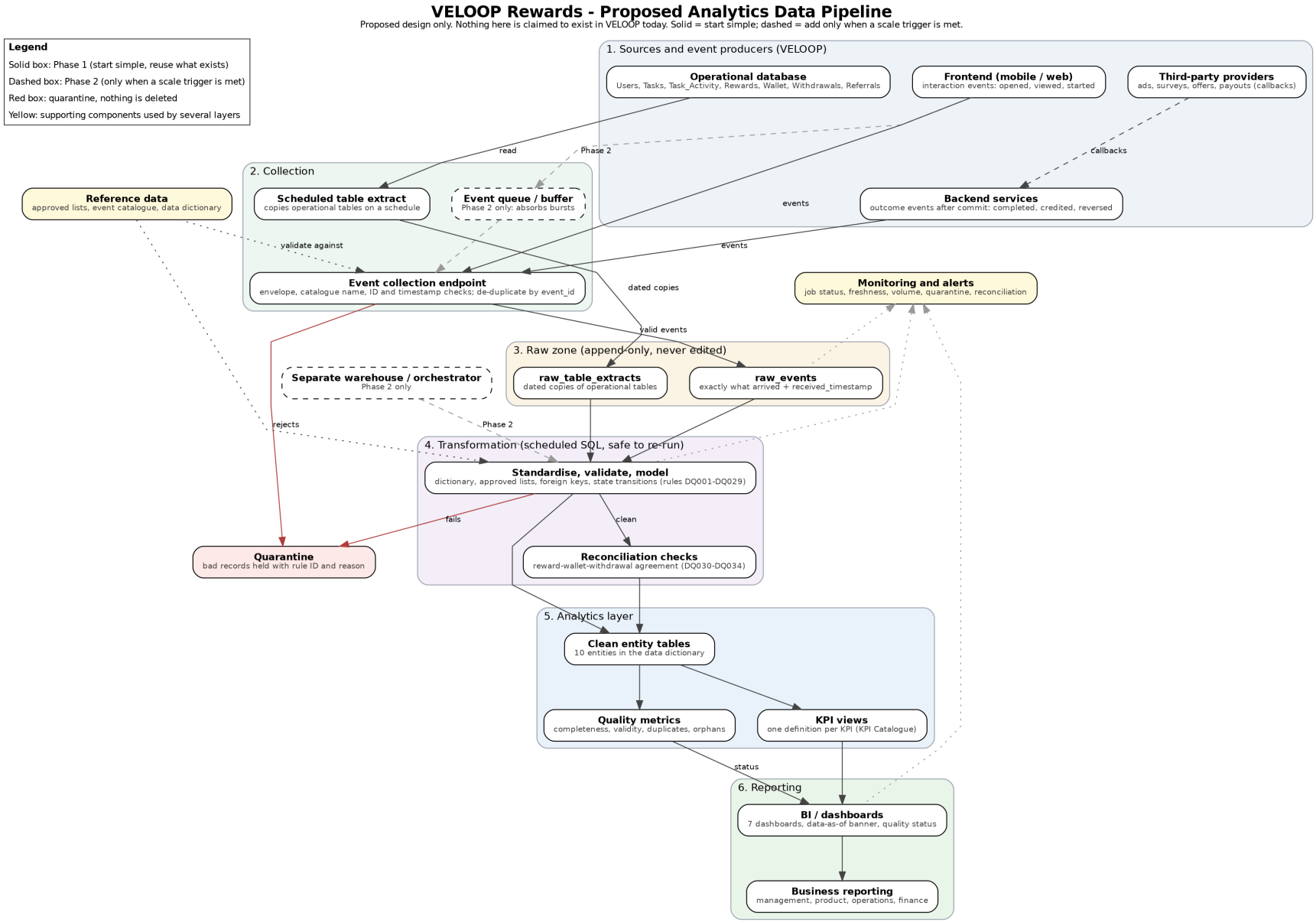
1. Summary

- **A practical pipeline in two phases.** Phase 1 reuses what VELOOP already runs: events and daily table extracts go to an append-only raw zone, scheduled SQL validates and models them, and dashboards read one analytics layer. Phase 2 components are added only when a named scale trigger is met (Section 9).
- **All 13 items the brief asks to document are covered** in the component table (Section 3): source systems, event producers, collection, storage, raw layer, transformation, quality checks, analytical layer, dashboard, refresh, monitoring, failure handling and ownership.
- **Nothing is chosen for appearance.** Tool examples are kinds of option, and the choice depends on VELOOP's existing stack, which is not yet known (A011).

2. Design principles

- Reuse before buying: start with what VELOOP already runs and add a component only when a named trigger is met.
- Keep raw data untouched and append-only, so any result can be rebuilt and any correction traced.
- Every transformation is safe to re-run, so a failed job is fixed by running it again, not by editing data.
- One definition per KPI: dashboards read the analytics layer, which implements the KPI Catalogue and nothing else.
- Quality is a pipeline stage, not a later clean-up: failing records are quarantined with a rule ID, never deleted.
- Simulated and production data stay in separate zones and are never silently mixed.
- Fewer moving parts: each extra tool adds cost, maintenance and failure points, so it needs an owner and a reason.

Architecture diagram



A vector version is in 16_Pipeline_Architecture_Diagram.pdf.

3. Components

Component	What It Does	Phase 1: start simple	Phase 2: only when justified	Inputs	Outputs	Owner	Status
Source systems	The places data originates	Use what exists: the app, backend services, the operational database and third-party providers. Their current state is unknown (A011)	Add sources only as new features launch	User and system activity	Events and operational tables	Engineering	To Be Verified
Event producers	Emit events from the app	Backend emits outcome events after commit; frontend emits a small set of interaction events; provider callbacks are received by the backend and emitted from there	Client SDK with an offline queue	Business actions	Events with the common payload	Engineering	Proposed
Collection	Receive and check events	One collection endpoint (or a write to an events table by the backend) that checks the common payload, catalogue name, ID and timestamp, and de-duplicates on event_id	A managed queue or stream to absorb bursts	Events	Valid events to raw; rejects to quarantine	Engineering	Proposed
Storage	Where analytics data lives	A separate schema or database from production tables, using the database engine VELOOP already runs; regular backups	A dedicated warehouse, with object storage for raw files	Valid events, table extracts	Stored raw, clean and analytics data	Engineering / Data	Proposed
Raw layer	Hold exactly what arrived	raw_events and dated raw_table_extracts; append-only; received_timestamp added; never edited	Raw files partitioned by date	Collection output, table extracts	Untouched history	Data Analyst	Proposed
Transformation	Standardise, validate and model	Scheduled SQL scripts or views run in a fixed order; each safe to re-run	A SQL modelling tool with tests and generated documentation; an orchestrator	Raw layer, reference data	Clean entity tables; quarantine	Data Analyst	Proposed
Quality checks	Apply the quality rules	One SQL check per rule ID (DQ001 to DQ039) writing to a results table and the quarantine table	Automated test framework with alert integration	Raw and clean data	Pass or fail per rule; quarantined records	Data Analyst / Engineering	Proposed
Analytical layer	Business-ready tables	Clean entity tables plus views that implement each KPI formula	Materialised tables where views are slow	Clean tables	Tables and KPI views for dashboards	Data Analyst	Proposed
Dashboard	Show results	Any BI option that reads only the analytics views (a spreadsheet, an open-source BI tool or a hosted one); decision pending	Governed BI with row-level security	Analytics layer	7 dashboards	Data Analyst / Product	To Be Verified
Refresh	Keep data current	Daily batch in a fixed run order (see run order)	More frequent refresh only for a dashboard where a named decision needs it	Schedule	Fresh tables and 'data as of' time	Data Analyst	Proposed
Monitoring	Notice problems early	A job log, a freshness check, a volume check, quarantine counts and reconciliation results summarised in one pipeline health view	Alerting tool with on-call routing	Job logs, rule results	Alerts to owners; health view	Engineering / Data	Proposed
Failure handling	Recover safely	Re-run idempotent jobs; dashboards show the last good data with a banner; a short runbook per failure	Automatic retries	Failure signals	Recovered data; incident note	Engineering / Data	Proposed
Ownership	Make someone responsible	Named owner per component, a RACI table and a change log (Work Package Q)	A data team with defined roles	Roles	Accountability	Product / Management	Proposed

4. Data flows

Flow	From	To	Content	Method (Phase 1)	Frequency	Notes
F1 Events	Frontend, backend	Collection, then raw_events	Feature events with the common payload	HTTP endpoint or events table	As they happen (batched delivery allowed)	Rejects go to quarantine
F2 Table extracts	Operational database	raw_table_extracts	Users, Tasks, Task_Activity, Rewards, Wallet, Withdrawals, Referrals, Support	Scheduled read from a replica or export, never from the live write path where avoidable	Daily	Dated copies keep history; no edits
F3 Provider callbacks	Third-party providers	Backend, then F1	Provider status and reference id	Backend receives the callback and emits provider_callback_received	As they happen	Missing callbacks stay 'unconfirmed' until the agreed wait passes
F4 Reference data	Product and Operations	Collection and transformation	Approved lists, event catalogue, data dictionary	One shared reference table or file under version control	On change	One source for app and pipeline (AG-10)
F5 Quarantine	Collection and transformation	Quarantine table	Records failing a rule, with rule ID and reason	Insert with raw reference	With each run	Triage owner assigned (open question)
F6 Metrics and alerts	All stages	Pipeline health view; owners	Job status, freshness, volume, quarantine and reconciliation results	Results table and summary message	Each run	Thresholds to be agreed

5. Data zones

Zone	Contents	Can be changed?	Who reads	Retention
Simulated (separate)	Simulated test datasets, each labelled 'SIMULATED DATA - CREATED FOR ANALYTICAL TESTING'	Rebuilt from documented generation method	Data Analyst, Intern (approved datasets)	Until replaced by governed production data; never merged silently
Raw	raw_events, raw_table_extracts	Never; append-only	Engineering, Data Analyst	To be agreed with Product (privacy)
Quarantine	Records that failed a rule	Status and resolution notes only	Data Analyst, Engineering	Until resolved, then per policy
Clean	Standardised, validated entity tables	Rebuilt by transformation	Data Analyst	To be agreed
Analytics	Entity tables and KPI views	Rebuilt by transformation	Data Analyst; aggregated to Product and Operations	To be agreed
Reporting	Dashboard extracts and shared reports	Rebuilt on refresh	Per access model (Work Package P)	Per report

6. Refresh and run order

Daily batch. Real-time is not proposed; it can be added for a dashboard where a named decision needs it.

Step	Job	Depends On	If It Fails
1	Ingest events and table extracts into the raw zone	-	Stop; alert; dashboards keep the last good data
2	Raw checks: freshness, late arrivals, volume (DQ036 to DQ038)	1	Pause downstream steps and alert
3	Transform and validate (DQ001 to DQ029); quarantine failures	2	Quarantine bad rows; fail the job only on a structural break; re-run after the fix
4	Reconciliation checks (DQ030 to DQ034)	3	Alert owners; flag the affected KPIs on dashboards
5	Build the analytics layer and KPI views	4	Keep the previous version; alert
6	Refresh dashboards and set the 'data as of' banner	5	Show the last good data with a warning banner
7	Publish the pipeline health summary	1 to 6	Alert Engineering if it cannot publish

7. Monitoring

Monitor	What It Measures	Alert Condition	Alert Goes To	Related Rule or KPI
Job status	Did each step finish?	Any step fails or does not start	Engineering, Data Analyst	Run order
Freshness	Age of the latest received event and extract	Older than the agreed limit (to be agreed)	Engineering	DQ036
Volume	Rows received per day versus the recent pattern	Outside the agreed band (to be agreed)	Engineering	DQ038
Late arrivals	Delay between event and receipt	Above the agreed tolerance (to be agreed)	Engineering	DQ037, KPI T03
Quarantine	Records held per rule	Any Critical rule hit, or a rising rate	Data Analyst	Quality metrics
Reconciliation	Reward, wallet and withdrawal agreement	Any exception	Data Analyst, Finance	DQ030 to DQ032, KPI T01
Schema drift	Unknown event names, versions or columns	Any appearance	Engineering	DQ020, DQ021, DQ023
Provider reliability	Callbacks received within the agreed wait	Rising share missing	Engineering	KPI T02
Dashboard refresh	Did every dashboard update?	Any failure	Data Analyst	Run order step 6
Storage growth	Size of raw and analytics data	Growth beyond what the plan assumes (to be agreed)	Engineering	Scale triggers

8. Failure handling

Failure	How It Is Detected	Immediate Handling	Recovery	Owner
Collection endpoint unavailable	Delivery errors; volume drop	The app queues and retries; the user's action is never blocked (AG-05)	Events arrive on retry; check raw counts against the backend	Engineering
Duplicate events	event_id repeats (DQ019)	Keep the first; count and log the rest	None needed beyond the log	Engineering
Malformed or unknown events	Collection rules (DQ003 to DQ006, DQ020, DQ022)	Quarantine with rule ID	Fix the producer, then reprocess from quarantine	Engineering
Late events	Delay above tolerance (DQ037)	Flag; restate the affected period	Re-run transformation for that period	Data Analyst
Transformation job fails	Job status	Dashboards keep the last good data and show a banner	Fix and re-run; jobs are safe to repeat	Data Analyst
Operational extract fails	Missing dated copy	Alert; reuse the previous copy with a banner	Re-run the extract	Engineering
Provider callback missing	Provider reliability monitor	Record stays 'unconfirmed'	Reconcile with the provider report; mark tracking_failed after the agreed wait	Engineering
Wallet reconciliation break	DQ030 to DQ032	Alert Finance and Engineering; flag affected KPIs	Investigate the entries; correct with a reversal or adjustment entry, never an edit	Data Analyst
Bad data discovered after publishing	Report or quality check	Withdraw or caveat the figure	Rebuild from the raw zone	Data Analyst
Schema change from a release	Schema drift monitor	Unknown versions quarantined	Update the catalogue and rules, then reprocess	Engineering / Data
Simulated data enters production	DQ039	Block the load	Remove and reload from the governed source	Data Analyst

Ownership (RACI)

R responsible, A accountable (one per row), C consulted, I informed.

Responsibility	Engineering	Data Analyst	Product	Operations / Finance
Event emission and instrumentation	A/R	C	C	I
Event catalogue and definitions	C	R	A	C
Approved value lists	R	C	A	C
Collection and raw storage	A/R	C	I	I
Transformation and clean tables	C	A/R	I	I
Quality rules and quarantine triage	R	A	I	C
KPI definitions	I	R	A	C
Dashboards	I	R	A	C
Pipeline monitoring and alerts	R	A	I	I
Access control and privacy	R	C	A	C
Change log	C	R	A	I

9. Scale triggers and cost principles

Trigger	Signal	Response (Phase 2 step)
Analytics queries slow the production database	Production slowdown that lines up with analytics runs	Move analytics to a separate database or warehouse
Events are lost in bursts	Gaps between backend records and raw events	Add a queue or buffer
Daily jobs overrun the refresh window	Dashboards not ready when needed	Make builds incremental; add an orchestrator
A decision needs fresher data	A named decision cannot wait for next-day data	Raise refresh for that dashboard only
Several people change the models	Conflicting or unreviewed changes	Use version control and review; a SQL modelling tool
Roles need row-level separation	Access model needs it (Work Package P)	Governed BI with role-based access
Provider data must be merged regularly	Manual provider reconciliation takes recurring effort	Dedicated provider ingestion

Cost principles

- Reuse the existing database, hosting and tools first.
- Prefer free, open-source or already-licensed tools for Phase 1.
- Every paid or managed component must point to a trigger above and have a named decision owner.
- Count maintenance, not only purchase cost: fewer tools means fewer failures to look after.
- No component is added because it sounds professional.

Tool examples (kinds of option, not a decision)

Layer	Phase 1 (example kind)	Phase 2 (example kind)
Collection	An endpoint in the existing backend, or writes to an events table	A managed event collector or queue
Storage	A separate schema in the database VELOOP already runs	A cloud warehouse with object storage
Transformation	SQL scripts and views run by a scheduler	A SQL modelling tool with tests, plus an orchestrator
Dashboard	A spreadsheet or an open-source BI tool	A hosted or governed BI service

10. Build sequence

Milestone	Outcome	Depends On	Backlog link
M1	Stable user_id, transaction_id, UTC timestamps and the common payload agreed	Engineering decisions (AP-01, AP-02, AP-10)	Critical pre-conditions
M2	Collection endpoint and raw zone receiving Critical events	M1	Event tracking, AG-01 to AG-04
M3	Transformation, quality rules and quarantine running for Critical events	M2, approved lists	Data quality checks
M4	Analytics layer and first dashboards: Executive Overview, Withdrawals, Wallet	M3	Dashboard blueprint
M5	Monitoring and the pipeline health view	M3	Monitoring
M6	High priority events, remaining dashboards	M4	Event tracking, dashboards
M7	Review scale triggers; add Phase 2 components only where triggered	M5	Scale triggers

11. Simulated and production data

- Simulated datasets live in their own zone and are labelled 'SIMULATED DATA - CREATED FOR ANALYTICAL TESTING'. Their generation method is documented.
- When production data becomes available, it is loaded through the governed pipeline and replaces the simulated data. The two are never silently mixed (rule DQ039).

12. Open questions

- What database, hosting and tools does VELOOP already run? This decides what Phase 1 reuses (A011).
- Which BI option will be used? The design is tool-neutral.
- Who owns triage of the quarantine table and how often?
- What freshness, volume and delay limits apply? (None are set here.)
- Can operational tables be read from a replica or export without affecting the live product?
- What retention period applies to raw data under the privacy rules? (Work Package P)

Next: Work Package P - Data Access, Privacy and Governance.